

WUFFI

Arquitectura, esquema Firestore y roadmap de implementación

Documento de diseño previo a la implementación

Versión 1.0 · Junio 2026

Tabla de contenidos

1. Visión del producto
 2. Arquitectura de información
 3. Mapa de navegación
 4. Esquema Firestore
 5. Modelos TypeScript y Zod
 6. Capa de servicios y estado
 7. Internacionalización
 8. Design system
 9. Configuración Firebase
 10. Fases de implementación
 11. Decisiones abiertas
 12. Checklist de aprobación
-

1. Visión del producto

WUFFI es una aplicación móvil con dos propósitos principales:

1. **Registro personal de mascotas** — Cuaderno digital privado para que cada usuario gestione el perfil de sus mascotas.
2. **Explorador público de mascotas** — Plataforma para publicar, explorar y colaborar con casos de mascotas perdidas, encontradas y en adopción.

La app debe sentirse **linda, moderna, amigable y confiable** — no solo una app de emergencia.

1.1 Dominios del producto

Dominio	Entidad	Visibilidad	Propósito
Cuaderno digital	Pet	Privada por defecto	Perfil personal del dueño
Alerta pública	Case (lost / found / adoption)	Pública	Explorador, mapa, contacto
Evidencia ligera	Sighting	Pública (vinculada a caso)	Avistamiento sin post completo
Identidad pública segura	QRProfile	Configurable	QR + contacto de emergencia
Organización	Organization	Pública	Refugios, ONGs, rescatistas

1.2 Regla central de datos

pets ≠ **cases**

- **pets** = perfiles privados o propiedad del usuario.
- **cases** = publicaciones públicas (perdida, encontrada, adopción).
- Un **Pet** puede generar varios **Case** a lo largo del tiempo.

Ejemplo: Nina es una mascota personal privada. Si Nina se pierde, se crea un **Case** tipo **lost** conectado al **petId** de Nina. Al ser encontrada, el caso pasa a **found** y se cierra; el perfil personal sigue existiendo.

1.3 Modos de perfil

Mascota personal (privada)

Campos: nombre, especie, raza, sexo, edad/fecha de nacimiento, peso, color, fotos, notas de personalidad, notas médicas, vacunas, alergias, contacto veterinario, microchip/ID, QR, contacto de emergencia.

No es pública por defecto. El usuario puede convertirla en alerta de mascota perdida.

Caso de mascota perdida (público)

Campos: datos de mascota, última ubicación vista, fecha, descripción, método de contacto, recompensa opcional, estado (active / sighted / found / closed), botones compartir/WhatsApp/llamar.

Puede crearse desde un perfil personal existente o desde cero.

Caso de mascota encontrada (público)

Campos: fotos, ubicación de hallazgo, fecha, descripción, estado de cuidado temporal, método de contacto, estado (looking_for_owner / reunited / closed).

Caso de adopción (público)

Camfields: datos de mascota, ubicación de adopción, estado de salud, vacunas, castrado/esterilizado, personalidad, requisitos, contacto, estado (available / reserved / adopted).

Publicable por usuarios, rescatistas, refugios u ONGs.

2. Arquitectura de información

2.1 Jerarquía de contenido

```
WUFFI
├─ Autenticación
│  ├─ Inicio de sesión (email, Google, Apple*)
│  ├─ Registro
│  └─ Recuperar contraseña
├─ Inicio (Home) - mis mascotas y acciones rápidas
│  ├─ Saludo + ubicación
│  ├─ Mis mascotas (cards)
│  ├─ Acciones rápidas
│  ├─ Alertas cercanas recientes
│  └─ Guardados recientes
├─ Explorar - descubrimiento público
│  ├─ Pestañas: Perdidas | Encontradas | Adopción
│  ├─ Vista mapa / lista
│  ├─ Filtros + búsqueda
│  └─ Detalle de caso
├─ Agregar - flujo multi-paso
│  ├─ ¿Qué querés agregar?
│  ├─ Mascota personal
│  ├─ Caso perdido (desde pet existente o nuevo)
│  ├─ Caso encontrado
│  └─ Caso adopción
├─ Alertas - notificaciones in-app
│  ├─ Por tipo (perdida, encontrada, avistamiento, adopción, interacción)
│  └─ Configuración (futuro push)
└─ Perfil
   ├─ Datos del usuario
   ├─ Mis mascotas
   ├─ Mis publicaciones
   ├─ Favoritos
   ├─ Organización (si aplica)
   └─ Ajustes (cuenta, notificaciones, privacidad)
```

*Apple Sign-In recomendado para App Store; incluido en arquitectura, prioridad de implementación según MVP.

2.2 Entidades y relaciones

User

- owns → Pet (1:N)
- creates → Case (1:N)
- reports → Sighting (1:N)
- saves → Favorite (1:N)
- receives → Notification (1:N)
- manages → Organization (0:1)

Pet

- has → QRProfile (1:1)
- has → PetHealthRecord (1:N)
- has → PetDocument (1:N)
- has → PetTimelineEvent (1:N)
- source for → Case (1:N, lost)

Case

- has → Sighting (1:N)
- has → CaseUpdate (1:N)

Organization

- publishes → Case adoption (1:N)

Favorite → references Case or Organization

2.3 Flujos principales de usuario

A. Primera mascota (onboarding implícito)

Home vacío → CTA "Agregar tu primera mascota" → Flujo personal multi-paso → Dashboard con card de mascota.

B. Mascota perdida desde perfil existente

Detalle de mascota → "Reportar como perdida" → Prefill con datos del pet → Ubicación, fecha, descripción → Publicar `Case` tipo `lost` vinculado al `petId`.

C. Explorar y contactar

Explorar → Filtros (especie, distancia, fecha, estado) → Mapa o lista → Bottom sheet preview → Detalle completo → WhatsApp / Llamar / Compartir / Guardar.

D. Avistamiento

Detalle de caso perdido → "Reportar avistamiento" → Foto opcional + ubicación + fecha/hora + notas → Crear `Sighting` vinculado → Notificación al dueño del caso.

E. QR de emergencia

Detalle de mascota personal → Pestaña QR → Configurar campos visibles públicamente → Generar/actualizar `QRProfile` → Escaneo muestra vista pública limitada (nombre, foto, contacto de emergencia, alertas médicas según configuración).

2.4 Pantallas principales

Pantalla	Secciones / contenido
Home	Saludo, mis mascotas, acciones rápidas, alertas cercanas, guardados, empty state
Explorar	Switch Perdidas/Encontradas/Adopción, mapa, lista, filtros, búsqueda por ciudad/barrio
Detalle mascota personal	Galería, Info, Salud, Documentos, QR, Timeline
Detalle caso público	Galería, Detalles, Ubicación, Contacto, Actualizaciones
Crear	Wizard multi-paso según tipo elegido
Alertas	Lista por tipo, marcado leído/no leído
Perfil	Avatar, ciudad/zona, mis mascotas, mis casos, favoritos, ajustes
Organización	Logo, ubicación, contacto, redes, adopciones activas, verificado

3. Mapa de navegación

Stack tecnológico de navegación: Expo Router (file-based routing) + bottom tabs + stacks anidados.

3.1 Estructura de carpetas propuesta

```
wuffi/
├─ app/
│   ├── _layout.tsx          # Root: providers, auth gate, i18n
│   ├── index.tsx            # Redirect: auth → tabs | login
│   │
│   ├── (auth)/
│   │   ├── _layout.tsx
│   │   ├── login.tsx
│   │   ├── register.tsx
│   │   └── forgot-password.tsx
│   │
│   ├── (tabs)/
│   │   ├── _layout.tsx      # Bottom tabs (5)
│   │   ├── index.tsx       # Home
│   │   ├── explore.tsx     # Explore hub
│   │   ├── add.tsx         # Create entry point
│   │   ├── alerts.tsx      # Notifications inbox
│   │   └── profile.tsx     # Profile hub
│   │
│   ├── pet/
│   │   ├── [id].tsx         # Personal pet detail
│   │   └── [id]/edit.tsx
│   │
│   ├── case/
│   │   ├── [id].tsx         # Public case detail
│   │   └── [id]/sighting.tsx # Report sighting
│   │
│   ├── create/
│   │   ├── _layout.tsx      # Multi-step wizard shell
│   │   ├── index.tsx        # "¿Qué querés agregar?"
│   │   ├── personal/[step].tsx
│   │   ├── lost/
│   │   │   ├── source.tsx   # ¿Usar mascota existente?
│   │   │   └── [step].tsx
│   │   ├── found/[step].tsx
│   │   └── adoption/[step].tsx
│   │
│   ├── explore/
│   │   ├── map.tsx
│   │   └── filters.tsx
│   │
│   ├── organization/[id].tsx
│   ├── favorites.tsx
│   ├── settings/
│   │   ├── index.tsx
│   │   ├── notifications.tsx
│   │   └── account.tsx
│   │
│   └── qr/[wuffiId].tsx     # Public QR landing
└─ src/
```

```

├── components/           # Design system
├── features/             # Módulos por feature
├── services/             # Adapters Firebase
├── stores/               # Zustand
├── hooks/                # React Query hooks
├── schemas/              # Zod
├── types/                # TypeScript
├── constants/
├── utils/
├── i18n/
│   ├── index.ts
│   └── locales/
│       ├── es-AR.json    # default
│       └── en-US.json
├── firebase/
│   ├── firestore.rules
│   ├── firestore.indexes.json
│   └── storage.rules
├── .env.example
└── app.config.ts

```

3.2 Bottom tabs

Tab	Ruta	Rol
Inicio	(tabs)/index	Mis mascotas + acciones rápidas
Explorar	(tabs)/explore	Casos públicos (perdidas, encontradas, adopción)
Agregar	(tabs)/add	Punto de entrada al wizard de creación
Alertas	(tabs)/alerts	Bandeja de notificaciones in-app
Perfil	(tabs)/profile	Cuenta, ajustes, favoritos

3.3 Rutas de deep link

Deep link	Destino
wuffi:// case/{id}	Detalle de caso público
wuffi:// pet/{id}	Detalle de mascota (requiere auth + ownership)
wuffi:// qr/{wuffiId}	Perfil QR público
wuffi:// explore?type=lost	Explorador con filtro
wuffi:// create/lost?petId={id}	Crear caso perdido desde pet

3.4 Flujo de navegación (resumen)

```
Login/Register → (tabs)
Home → pet/[id] → create/lost/*
Home → create/*
Add tab → create/*
Explore → case/[id] → case/[id]/sighting
Explore → explore/map
Profile → favorites, settings/*, organization/[id]
Pet detail → QR tab → qr/[wuffiId] (público)
```

4. Esquema Firestore

4.1 Colecciones

Colección	ID documento	Descripción
users	{uid}	Perfil de usuario
pets	auto	Mascotas privadas del owner
cases	auto	Posts públicos lost/found/adoption
sightings	auto	Avistamientos
favorites	{uid}_{targetType}_{targetId}	Guardados
notifications	auto	Notificaciones in-app
organizations	auto	ONGs / refugios
documents	auto	Metadatos de archivos (Storage)
qrProfiles	{wuffiId}	Perfil QR público
reports	auto	Denuncias / moderación

Subcolección recomendada: `cases/{caseId}/updates` — historial de actualizaciones de un caso.

4.2 Documento: users/{uid}

```
{
  uid: string
  email: string
  displayName: string
  photoURL?: string
  phone?: string
  city?: string
  zone?: string
  geo?: GeoPoint // centro de zona para alertas
  locale: "es-AR" | "en-US"
  role: "user" | "org_admin" | "moderator"
  organizationId?: string
  notificationSettings: {
    lostNearby: boolean
    foundNearby: boolean
    sightings: boolean
    adoptionUpdates: boolean
    interactions: boolean
    pushEnabled: boolean
    radiusKm: number // default 10
  }
  pushToken?: string // FCM / Expo (fase 2)
  createdAt: Timestamp
  updatedAt: Timestamp
}
```

4.3 Documento: pets/{petId}

```
{
  id: string
  ownerId: string
  visibility: "private"

  name: string
  species: "dog" | "cat" | "rabbit" | "bird" | "other"
  breed?: string
  sex: "male" | "female" | "unknown"
  birthDate?: Timestamp
  ageMonths?: number
  weightKg?: number
  color?: string
  photoUrls: string[]
  personalityNotes?: string

  medicalNotes?: string
  vaccines?: {
    name: string
    date: Timestamp
    nextDue?: Timestamp
  }[]
  allergies?: string[]
  vetContact?: {
    name: string
    phone?: string
    email?: string
  }

  microchipId?: string
  customId?: string
  wuffiId: string // slug único para QR

  emergencyContact?: {
    name: string
    phone: string
    relationship?: string
  }

  status: "safe" | "lost"
  activeLostCaseId?: string

  createdAt: Timestamp
  updatedAt: Timestamp
}
```

4.4 Documento: cases/{caseId}

Campos comunes a todos los tipos:

```

{
  id: string
  caseType: "lost" | "found" | "adoption"
  ownerId: string
  petId?: string
  organizationId?: string

  petSnapshot: {
    name: string
    species: string
    breed?: string
    sex?: string
    ageMonths?: number
    weightKg?: number
    color?: string
    photoUrls: string[]
  }

  title: string
  description: string
  status: string // varía por caseType

  location: GeoPoint
  locationGeoHash: string
  addressText?: string
  city?: string
  neighborhood?: string

  contact: {
    showPhone: boolean
    showWhatsApp: boolean
    phone?: string
    whatsapp?: string
    preferredMethod: "whatsapp" | "phone" | "in_app"
  }

  sightingCount: number
  favoriteCount: number

  createdAt: Timestamp
  updatedAt: Timestamp
  closedAt?: Timestamp
}

```

Extensión lost:

```
{
  caseType: "lost"
  status: "active" | "sighted" | "found" | "closed"
  lastSeenAt: Timestamp
  lastSeenLocation: GeoPoint
  reward?: {
    amount?: number
    currency?: "ARS"
    description?: string
  }
}
```

Extensión found:

```
{
  caseType: "found"
  status: "looking_for_owner" | "reunited" | "closed"
  foundAt: Timestamp
  foundLocation: GeoPoint
  temporaryCare: "with_finder" | "shelter" | "street" | "vet"
}
```

Extensión adoption:

```
{
  caseType: "adoption"
  status: "available" | "reserved" | "adopted"
  adoptionLocation: GeoPoint
  healthStatus?: string
  vaccinated: boolean
  neutered: boolean
  personality?: string
  requirements?: string[]
  size?: "small" | "medium" | "large"
}
```

4.5 Subcolección: cases/{caseId}/updates/{updateId}

```
{
  type: "status_change" | "sighting" | "comment" | "system"
  message: string
  authorId?: string
  metadata?: Record<string, unknown>
  createdAt: Timestamp
}
```

4.6 Documento: sightings/{sightingId}

```
{
  id: string
  caseId: string
  reporterId: string
  photoUrl?: string
  location: GeoPoint
  locationGeoHash: string
  seenAt: Timestamp
  notes?: string
  status: "pending" | "confirmed" | "dismissed"
  createdAt: Timestamp
}
```

4.7 Documento: favorites/{favoriteId}

```
{
  id: string // uid_targetType_targetId
  userId: string
  targetType: "case" | "organization"
  targetId: string
  caseType?: "lost" | "found" | "adoption"
  createdAt: Timestamp
}
```

4.8 Documento: notifications/{notificationId}

```
{
  id: string
  userId: string
  type:
    | "lost_nearby"
    | "found_nearby"
    | "sighting_update"
    | "adoption_update"
    | "interaction"
  titleKey: string
  bodyKey: string
  params?: Record<string, string>
  read: boolean
  deepLink?: string
  relatedId?: string
  createdAt: Timestamp
}
```

4.9 Documento: organizations/{orgId}

```
{
  id: string
  name: string
  logoUrl?: string
  location: GeoPoint
  addressText?: string
  city?: string
  contact: {
    phone?: string
    email?: string
    whatsapp?: string
  }
  socialLinks?: {
    instagram?: string
    website?: string
  }
  verified: boolean
  adminIds: string[]
  activeAdoptionCount: number
  createdAt: Timestamp
  updatedAt: Timestamp
}
```

4.10 Documento: documents/{docId}

```
{
  id: string
  petId: string
  ownerId: string
  type: "vaccine_card" | "medical" | "pedigree" | "other"
  title: string
  storagePath: string
  mimeType: string
  createdAt: Timestamp
}
```

4.11 Documento: qrProfiles/{wuffiId}

```
{
  wuffiId: string
  petId: string
  ownerId: string
  publicFields: {
    showPhoto: boolean
    showName: boolean
    showEmergencyContact: boolean
    showMedicalWarnings: boolean
    showAllergies: boolean
  }
  displayName?: string
  photoUrl?: string
  emergencyPhone?: string
  medicalWarnings?: string[]
  status: "safe" | "lost"
  lostCaseId?: string
  updatedAt: Timestamp
}
```


4.12 Documento: reports/{reportId}

```
{
  reporterId: string
  targetType: "case" | "user" | "sighting"
  targetId: string
  reason: string
  notes?: string
  status: "open" | "reviewed" | "actioned"
  createdAt: Timestamp
}
```

4.13 Firebase Storage — paths

```
/users/{uid}/avatar.jpg
/pets/{petId}/photos/{fileId}.jpg
/cases/{caseId}/photos/{fileId}.jpg
/sightings/{sightingId}/photo.jpg
/organizations/{orgId}/logo.jpg
/documents/{petId}/{docId}.pdf
```

4.14 Índices compuestos Firestore

Colección	Campos	Uso
<code>cases</code>	caseType ASC, status ASC, createdAt DESC	Listas por tipo
<code>cases</code>	caseType ASC, locationGeoHash ASC, createdAt DESC	Proximidad + tipo
<code>cases</code>	caseType ASC, city ASC, createdAt DESC	Búsqueda por ciudad
<code>cases</code>	ownerId ASC, createdAt DESC	Mis publicaciones
<code>cases</code>	organizationId ASC, status ASC	Adopciones ONG
<code>pets</code>	ownerId ASC, updatedAt DESC	Mis mascotas
<code>sightings</code>	caseId ASC, createdAt DESC	Avistamientos de un caso
<code>notifications</code>	userId ASC, read ASC, createdAt DESC	Bandeja alertas
<code>favorites</code>	userId ASC, createdAt DESC	Guardados

Geo queries: geohash con `geofire-common` + prefix queries sobre `locationGeoHash`. Alternativa futura: GeoFirestore extension o Algolia Geo.

4.15 Reglas de privacidad (borrador)

Recurso	Lectura	Escritura
users	Solo propio	Solo propio
pets	Solo owner	Solo owner
cases	Pública (todos)	Owner o org admin
sightings	Si caso es público	Auth + create; owner confirma
favorites	Solo propio	Solo propio
notifications	Solo propio	Sistema / Cloud Functions
organizations	Pública	adminIds
documents	Solo owner	Solo owner
qrProfiles	Pública (campos limitados)	Solo owner
reports	Moderadores	Auth create
Storage	Por path + ownership	Por path + ownership

Principios:

- Mascotas personales son privadas por defecto.
 - Casos públicos visibles para todos.
 - El usuario decide qué contacto mostrar en casos públicos.
 - QR solo expone campos habilitados por el dueño.
 - Documentos de salud nunca se exponen públicamente.
-

5. Modelos TypeScript y Zod

5.1 Discriminated unions

```
type Case =
  | (BaseCase & { caseType: "lost"; status: LostStatus; ... })
  | (BaseCase & { caseType: "found"; status: FoundStatus; ... })
  | (BaseCase & { caseType: "adoption"; status: AdoptionStatus; ... })

type LostStatus = "active" | "sighted" | "found" | "closed"
type FoundStatus = "looking_for_owner" | "reunited" | "closed"
type AdoptionStatus = "available" | "reserved" | "adopted"
type PetVisibility = "private"
type QRPublicStatus = "safe" | "lost"
type Species = "dog" | "cat" | "rabbit" | "bird" | "other"
type Sex = "male" | "female" | "unknown"
```

5.2 Entidades con schema Zod

Entidad	Archivo types	Archivo schema
User	types/user.ts	schemas/user.schema.ts
Pet	types/pet.ts	schemas/pet.schema.ts
PetHealthRecord	types/pet-health.ts	schemas/pet-health.schema.ts
PetDocument	types/pet-document.ts	schemas/pet-document.schema.ts
PetTimelineEvent	types/pet-timeline.ts	schemas/pet-timeline.schema.ts
Case (union)	types/case.ts	schemas/case.schema.ts
Sighting	types/sighting.ts	schemas/sighting.schema.ts
Organization	types/organization.ts	schemas/organization.schema.ts
Favorite	types/favorite.ts	schemas/favorite.schema.ts
Notification	types/notification.ts	schemas/notification.schema.ts
QRProfile	types/qr-profile.ts	schemas/qr-profile.schema.ts

5.3 Mappers Firestore

Cada servicio incluirá conversores `firestoreToDomain()` y `domainToFirestore()` para timestamps, GeoPoint y enums.

6. Capa de servicios y estado

6.1 Stack

Capa	Tecnología	Responsabilidad
UI	React Native + NativeWind	Pantallas y design system
Navegación	Expo Router	Tabs, stacks, deep links
Server state	React Query (@tanstack/react-query)	Cache, mutations, optimistic updates
Client state	Zustand	Auth session, filtros mapa, draft wizard
Backend	Firebase Auth, Firestore, Storage	Fuente de verdad
Validación	Zod	Forms multi-step
i18n	i18next + expo-localization	es-AR default
Mapas	react-native-maps + expo-location	Markers, geolocation
Push (prep)	expo-notifications + FCM	Placeholder fase 2

6.2 Estructura de servicios

```
src/services/  
├─ firebase/  
│   ├── app.ts           # Init Firebase  
│   ├── auth.ts  
│   ├── firestore.ts  
│   └─ storage.ts  
├─ users/userService.ts  
├─ pets/petService.ts  
├─ cases/caseService.ts  
├─ sightings/sightingService.ts  
├─ favorites/favoriteService.ts  
├─ notifications/notificationService.ts  
├─ organizations/organizationService.ts  
├─ qr/qrProfileService.ts  
└─ geo/geoQueryService.ts
```

6.3 Patrón de hooks (React Query)

```
hooks/
├─ useAuth.ts
├─ usePets.ts
├─ usePet.ts
├─ useCases.ts
├─ useCase.ts
├─ useExploreCases.ts
├─ useSightings.ts
├─ useFavorites.ts
├─ useNotifications.ts
└─ useOrganizations.ts
```

6.4 Stores Zustand

```
stores/
├─ authStore.ts          # session, user profile cache
├─ exploreStore.ts       # active tab, filters, map region
├─ createWizardStore.ts  # multi-step form draft
└─ uiStore.ts            # bottom sheets, modals
```

7. Internacionalización

Idioma primario: Español (Argentina) — `es-AR`

Arquitectura: i18n desde día uno; sin strings hardcodeados en componentes.

7.1 Estructura

```
src/i18n/
├─ index.ts
├─ config.ts
└─ locales/
    ├─ es-AR.json      # default
    └─ en-US.json      # stub / futuro
```

7.2 Ejemplos de keys

```
{
  "tabs.home": "Inicio",
  "tabs.explore": "Explorar",
  "tabs.add": "Agregar",
  "tabs.alerts": "Alertas",
  "tabs.profile": "Perfil",
  "home.greeting": "¡Hola, {{name}}!",
  "home.myPets": "Mis mascotas",
  "home.empty.title": "Todavía no tenés mascotas",
  "home.empty.cta": "Agregar mascota",
  "home.quickActions.addPet": "Agregar mascota",
  "home.quickActions.reportLost": "Reportar mascota perdida",
  "home.quickActions.addFound": "Agregar mascota encontrada",
  "home.quickActions.addAdoption": "Publicar en adopción",
  "explore.lost": "Mascotas perdidas",
  "explore.found": "Mascotas encontradas",
  "explore.adoption": "En adopción",
  "case.contact.whatsapp": "Contactar por WhatsApp",
  "case.contact.call": "Llamar",
  "case.contact.share": "Compartir",
  "case.status.active": "Activa",
  "case.status.found": "Encontrada",
  "create.title": "¿Qué querés agregar?",
  "create.personalPet": "Mi mascota personal",
  "create.lostPet": "Mascota perdida",
  "create.foundPet": "Mascota encontrada",
  "create.adoption": "Mascota en adopción"
}
```

Expansión futura: portugués (pt-BR) agregando archivo JSON sin modificar pantallas.

8. Design system

Referencia visual: UI neo-brutalist suave — cute, cozy, pastel, bordes redondeados, outlines negros, sombras suaves.

8.1 Paleta de colores

Token	Hex	Uso
primary	#F9A23B	CTAs, tab activo, pins mapa, banners
background	#FFF4EA	Fondo general de la app
lavender	#D8C3FF	Chips, cards categoría, stats
pink	#FFC8D8	Badges, info cards
sky	#BDEFFF	Found, info secundaria
mint	#CFF5DC	Adopción, salud
text	#1F2937	Títulos y texto principal
muted	#6B7280	Subtítulos, placeholders
card	#FFFFFF	Superficies de cards
border	#000000	Outline 1.5–2pt en cards interactivas

8.2 Tipografía y forma

- Sans-serif amigable (Inter o similar via Expo Google Fonts).
- Border radius: 16–24px (`rounded-2xl` / `rounded-3xl`).
- Bordes: `border-2 border-black` en cards, inputs, tabs flotantes.
- Sombras suaves en cards principales.

8.3 Componentes base

Componente	Descripción
Card	Superficie blanca redondeada con borde
PetCard	Card grande con foto 3D-style, favorito, título
CaseCard	Card de caso público con status badge
Chip	Filtro/categoría con color pastel
StatusBadge	Pill con estado del caso
BottomSheet	Preview en mapa y acciones rápidas
EmptyState	Ilustración + CTA para onboarding
PrimaryButton	Naranja, full-width, borde negro
SearchBar	Input redondeado con ícono
MapMarker	Pin por tipo (lost/found/adoption/sighting)
PhotoGallery	Carrusel en detalle
FloatingTabBar	Tab bar blanca flotante con borde

8.4 Markers en mapa

Tipo	Color sugerido	Ícono
Lost	Primary orange	Pin perdida
Found	Sky blue	Pin encontrada
Adoption	Mint green	Pin adopción
Sighting	Lavender	Pin avistamiento

Bottom sheet en marker: foto, nombre, estado, distancia, botón principal.

9. Configuración Firebase

9.1 Servicios a habilitar

1. **Firebase Authentication** — Email/Password, Google Sign-In; Apple preparado para iOS.

2. **Cloud Firestore** — Región `southamerica-east1` (São Paulo, más cercana a Argentina).
3. **Firebase Storage** — Fotos, avatares, documentos.

9.2 Archivos de configuración (post-aprobación)

```
wuffi/
├─ google-services.json          # Android
├─ GoogleService-Info.plist     # iOS
├─ .env.example
├─ .env                          # gitignored
├─ app.config.ts                # EXPO_PUBLIC_FIREBASE_*
└─ firebase/
    ├─ firestore.rules
    ├─ firestore.indexes.json
    └─ storage.rules
```

9.3 Variables de entorno

```
EXPO_PUBLIC_FIREBASE_API_KEY=
EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN=
EXPO_PUBLIC_FIREBASE_PROJECT_ID=
EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET=
EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=
EXPO_PUBLIC_FIREBASE_APP_ID=
EXPO_PUBLIC_GOOGLE_MAPS_API_KEY=
```

9.4 Principio de datos

Firebase es el backend primario. No usar mock data si Firebase está configurado. La capa de servicios abstrae Firestore para permitir tests y fallback durante desarrollo local sin conexión.

10. Fases de implementación

Fase 1 — MVP (implementación completa)

#	Feature	Entregable
1	Scaffold	Expo + TypeScript + Expo Router + NativeWind + i18n
2	Firebase	Auth, Firestore rules, Storage rules, env config
3	Auth	Login, registro, logout, auth gate
4	Mascota personal	CRUD pet, fotos, Home dashboard
5	Wizard crear	Flujo multi-paso: personal / lost / found / adoption
6	Casos públicos	CRUD cases, prefill desde pet existente
7	Explorar	Lista + switch lost/found/adoption + filtros
8	Mapa	Markers, geolocation, bottom sheet preview
9	Detalle	Pantalla caso + pantalla mascota personal (tabs)
10	Contacto	WhatsApp, llamar, compartir deep link
11	Favoritos	Guardar/quitar casos y orgs
12	Perfil	Datos usuario, mis mascotas, mis casos, ajustes
13	Alertas in-app	Modelo + UI bandeja (sin push real)

Fase 2 — Arquitectura preparada, implementación incremental

Feature	Descripción
Push notifications	FCM + expo-notifications, tokens en users
QR system	Generación, configuración privacidad, landing pública
ONGs	Perfiles verificados, adopciones vinculadas
Avistamientos	Flujo completo + notificación al dueño
Moderación	reports + revisión
Cloud Functions	Alertas geo, contadores denormalizados

Fase 3 — Roadmap futuro

- Marketplace de productos para mascotas

- Reconocimiento AI de raza/foto
- Directorio veterinario
- Gamificación (badges, contribuciones)
- Web companion para QR profiles

11. Decisiones abiertas

#	Decisión	Propuesta	Alternativa
1	Ubicación del código	Carpeteta/repo <code>wuffi/</code> separado de Mochileaf	Monorepo
2	Login social MVP	Email + Google	+ Apple desde día 1
3	Geo queries	geohash + índices compuestos	GeoFirestore extension
4	QR público	Deep link + página web futura	Solo in-app
5	Idioma secundario MVP	en-USjson stub	Solo es-AR
6	Organizaciones MVP	Modelo + perfil básico sin verificación manual	Postergar ONGs

12. Checklist de aprobación

Marcar antes de iniciar implementación:

- ☐ Separación `pets` / `cases` / `sightings`
- ☐ Estructura de navegación Expo Router
- ☐ Esquema Firestore y reglas de privacidad
- ☐ Estados por tipo de caso
- ☐ i18n es-AR + translation keys (sin strings en componentes)
- ☐ Design tokens y dirección visual
- ☐ Alcance Fase 1 MVP vs features diferidas
- ☐ Decisiones abiertas (#1—#6)

Tech stack confirmado

Área	Tecnología
Framework	React Native + Expo
Lenguaje	TypeScript
Routing	Expo Router
Auth	Firebase Authentication
Database	Cloud Firestore
Files	Firebase Storage
Mapas	React Native Maps
Estado cliente	Zustand
Estado servidor	React Query
Validación	Zod
Estilos	NativeWind (Tailwind CSS)
i18n	i18next + expo-localization

WUFFI · Arquitectura v1.0 · Junio 2026

Documento generado para revisión y aprobación antes de la implementación.